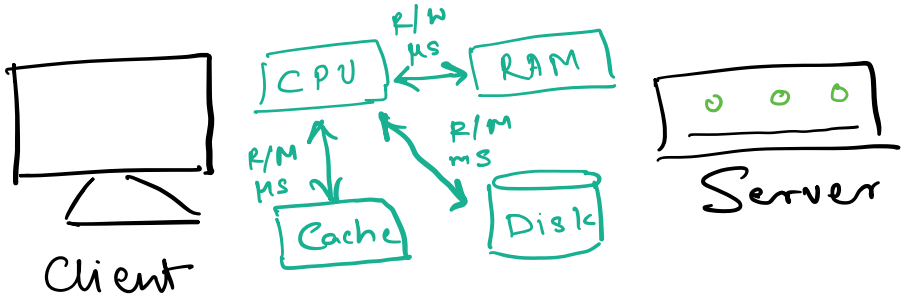
# Caching



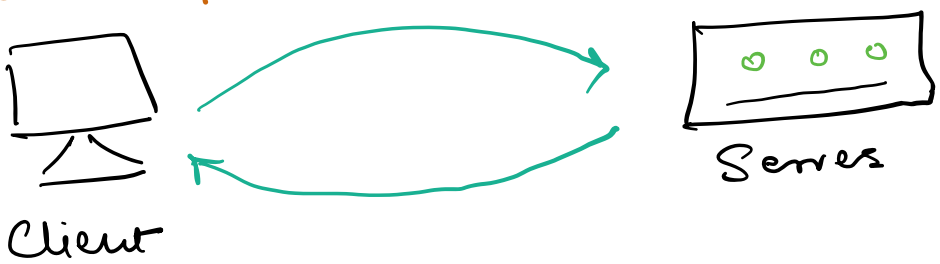

⁂ Caching is taking a copy of data that already exist in RAM and put it in cache since Read/Write is faster in cache so as to

↓ Latency ↑ Throughput

↓ Network cost (Reduces sending same data on the network by pulling through server instead having it stored in nearby cache)

⁂ Example leetcode.io the get main page content is static and so it can be cached in our computer. If done that then there wont be any network call to the server.

1) Client's side Caching




Check

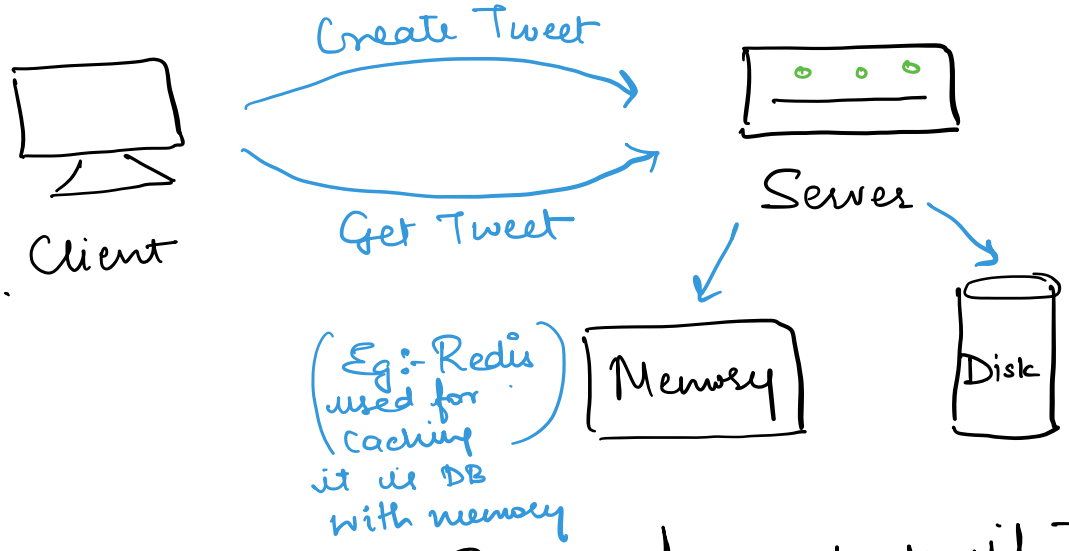
Cache hit – found data

Cache miss – Not found data

Cache Ratio – $\frac{\text{hit}}{\text{hit + miss}}$


→ Disk can still be considered a cache in context to making a network call to server as that could cost 0.1 to some seconds while disk can take ms.

## 2) Server side caching




* Get a tweet so Server first check if that tweet is in Memory if no then will go to disk. Then take that tweet from disk and return to user but also store it in memory for next time
Eg:- the most popular/frequented tweets can be stored in Cache/ memory to speed up the performance
∴ ↓ latency ↑ throughput

In short, in write the server will directly write in the disk and then in Read check memory first it will be cache miss then Check disk store that copy in memory & return

write-around Caching →

Write-through Caching → Creating a resource like tweet we immediately write it out in the memory & also to permanent disk storage

Write-Back Caching (Faster) → Creating a tweet and writing it only in memory/cache but missing it from disk. So next time a read request comes

can get data directly from cache
however it does save data later to
disk periodically. Meanwhile if system
crashes not reliable as might lead
to some data loss permanently

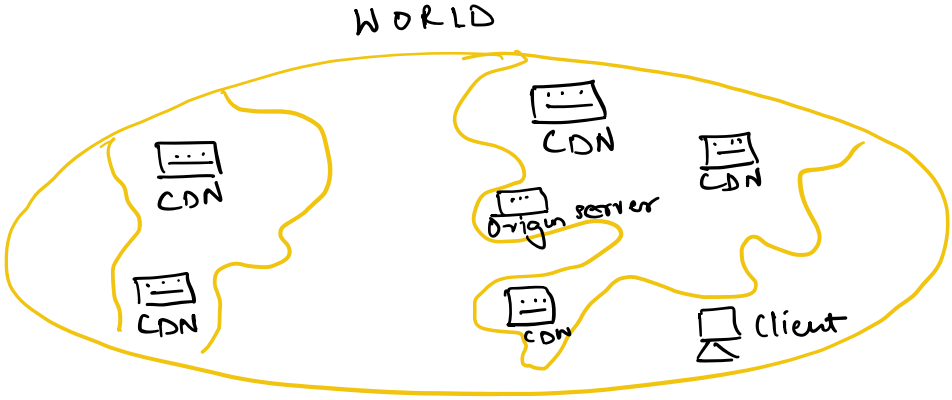
## Eviction Policy (In case the cache is full which one should be removed first)

First in First out
1) FIFO

Time Range latest needed
2) LRU (Tweets ✓)

More popular needed
3) LFU (Tweets)

## Content Delivery Network (CDN)




CDN/Cache - is a way we can cache data closer to the end user (can only get static content eg image/videos)
Incase of crash user can go to other CDN ↑ Availability Reliability
Origin Server - can run the application code/ dynamic content

CDN
Push... Pull...

1) Push CDN Twitter knew its image (static) the Push CDN will push the data to the origin server that will also push it to every other CDN so the user can access the closest CDN server don't have to go to the original server
eg:-

2) Pull CDN
eg:- Traditional CDN if user upload a profile picture on Twitter than it will be available in the origin server but wont push it to the CDN Server immediately. User makes req to see the profile picture it first checks the CDN if not there than CDN (acts as a proxy) CDN will go to the origin server return it to CDN & so CDN will now cache the req and return it to user so next time any one checks the image in CDN it will be CACHE HIT

※ Does not cache the same data / profile pic to all the other CDN

Note:- In Header if Cache control is PUBLIC for a given JS/static file than it can be cached by a CDN Server (PRIVATE cache control cannot be cached by a CDN Server)

=

Is Relevant for Pull CDN